

FULL STACK DEVELOPMENT

*A Summer Internship Report submitted in partial fulfillment of the
requirements for the award of degree of*

BACHELOR OF TECHNOLOGY

in

CSE - Data Science

Submitted by

CHITTUDI UMA MAHESWARA RAO

22P31A4405



CSE - Data Science

ADITYA ENGINEERING COLLEGE & TECHNOLOGY (A)

Approved by AICTE, Permanently affiliated to JNTUK & Accredited by NAAC with 'A+' Grade

Recognized by UGC under the sections 2(f) and 12(B) of the UGC act 1956

Aditya Nagar, ADB Road –Surampalem 533437, E.G. Dist., A.P.,

2025-2026

ADITYA ENGINEERING COLLEGE & TECHNOLOGY (A)

Approved by AICTE, Permanently Affiliated to JNTUK & Accredited by NAAC with 'A+' Grade

Recognized by UGC under the sections 2(f) and 12(B) of the UGC act 1956

Aditya Nagar, ADB Road - Surampalem – 533437, E.G.Dist., A.P.,

Department Of CSE - Data Science



CERTIFICATE

This is to certify that the Internship report entitled “**FULL STACK DEVELOPMENT WITH REACT NATIVE**” is being submitted By **CHITTUDI UMA MAHESWARA RAO** Roll no: **22P31A4405**. In partial fulfillment of the requirements for award of the B. Tech degree in CSE – Data Science for the academic year 2025- 2026.

Internship Coordinator

Mrs. Talluri Sushma ,M.Tech,(Ph.D.)
Assistant Professor
CSE-Data Science

Head of the Department

Dr.B.ManikyalaRao,M.Tech.,Ph.D
Associate Professor
CSE-Data Science

DECLARATION

I hereby declare that the Internship entitled “**FULL STACK DEVELOPMENT WITH REACT NATIVE**” is a genuine report. This work has been submitted to the **ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY (A)**, Surampalem, permanently affiliated to **JNTUK, KAKINADA** in partial fulfillment of the **B.Tech** degree.

I also hereby declare that this internship report not submitted in full or partial any other university for any degree.

CHITTUDI UMA MAHESWARA RAO
22P31A4405



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An AUTONOMOUS Institution

Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(f) and 12(b) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.
Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

INSTITUTE VISION AND MISSION

VISION:

To induce higher planes of learning by imparting technical education with

- International standards
- Applied research
- Creative Ability
- Value based instruction and to emerge as a premiere institute

MISSION:

Achieving academic excellence by providing globally acceptable technical education by forecasting technology through

- Innovative research and development
- Industry institute interaction
- Empowered manpower

PRINCIPAL

PRINCIPAL
Aditya College of
Engineering & Technology
SURAMPALEM- 533 437

INTERNSHIP COMPLETION CERTIFICATE



CERTIFICATE OF INTERNSHIP

Date: 28-07-2025

This is to certify that Mr. Chittudi Uma Maheswara Rao, of the Computer Science and Engineering(DS) department with Roll No: 22P31A4405 of Aditya College of Engineering & Technology(A) has successfully completed a summer internship with Technical Hub Pvt Ltd from 02-06-2025 to 28-07-2025.

During this tenure, the trainee worked with the technology Full Stack Development with React Native and excelled in major concepts.

- Cross-Platform Development
- Database Management
- Authentication & Authorization
- Security & Flexibility

The Trainee has a great amount of responsibility, sincerity and a genuine willingness to learn new things.

We found the trainee's performance and conduct were satisfactory.

We wish you all the best and success in your future endeavors.

Intern ID – THSI251446

<https://verify.technicalhub.io/>



+91 83 43 81 81 81
support@technicalhub.io

www.technicalhub.io



BABJI NEELAM
Founder & CEO

ACKNOWLEDGEMENT

I offer my sincere thanks to precious and dynamic Principal, **Dr. A. Ramesh**, M.Tech.,Ph.D, **Aditya College of Engineering & Technology (A)** for his co-operation.

My sincere thanks to **Dr.B.ManikyalaRao**,M.Tech.,Ph.D, Head of the department of CSE – Data Science for his valuable support.

I express my sincere gratitude to my internship guide, **Mrs. Talluri Sushma** ,M.Tech,(Ph.D.) **Assistant Professor, Department of CSE-Data Science**, for her valuable guidance and encouragement which has been helpful in successful completion of this internship.

With immense pleasure I would like to express my deep sense and heart full thanks to the management of **Aditya College of Engineering & Technology (A)**.

Learning Objectives/Internship Objectives

1. **To gain practical knowledge of Full Stack Development** using the MERN stack by building end-to-end web and mobile applications.
2. **To develop interactive front-end interfaces** using React.js for web and React Native for mobile platforms with a focus on user experience and responsiveness.
3. **To understand backend development** using Node.js and Express.js, including API creation, routing, and database integration with MongoDB.
4. **To learn real-world deployment and version control** practices using Git, GitHub, and platforms like Vercel or Expo.
5. **To enhance problem-solving and teamwork skills** by working on live projects, debugging issues, and collaborating in an agile development environment.

CONTENTS

CHAPTER 1: EXECUTIVE SUMMARY	Page No
1.1 Learning Objectives of Summer Internship	1
1.2 Learning Outcomes of Summer Internship	1
1.3 Summar Internship Summary	1
CHAPTER 2: OVERVIEW OF THE ORGANIZATION	
2.1 Introduction of the Organization	2
2.2 Misson of the Organization	2
2.3 Vision of the Organization	2
CHAPTER 3: INTERNSHIP PART	
3.1 A Discourse on Full Stack Development	3 - 4
3.1.1 An Exposition of the Mern Stack	
3.2 Configure of the Development Environment	4 - 5
3.2.1 Installation of Core Instrumentation	
3.3 Backend Development utilizing Node.js and Express.js	5 - 7
3.3.1 An Introduction to the Node.js Runtime	
3.3.2 The Construction of a RESTful API with Express.js	
3.3.3 A Conceptual Overview of Middleware	
3.3.4 Request Routing	
3.4 Database Administration with MongoDB	7 - 9
3.4.1 An Introduction to NoSQL Database Systems	
3.4.2 MongoDB Atlas and Mongoose	
3.4.3 The Creation of Schemas and Models	
3.4.4 CRUD Operations	
3.5 Frontend Development with the React Library	9 - 11
3.5.1 Core Concepts of React	
3.5.2 Initialization of a React Project	
3.5.3 State Management utilizing the Context API and Redux	
3.6 An Introduction to the React Native Framework	11-13
3.6.1 A Definition of React Native	

3.6.2 Salient Distinctions: React versus React Native	
3.6.3 Configuration of the React Native Environment	
3.7 The Construction of a Mobile Application with React Native	13-15
3.7.1 Core Components in React Native	
3.7.2 Styling in React Native	
3.7.3 Layout with Flexbox	
3.7.4 Navigation utilizing React Navigation	
3.7.5 Accessing Native Device Features	
3.9 Conclusion	16
CHAPTER 4: WEEKLY REPORTS	
4.1 Activity log for the First week	17
4.1.1 Weekly Report of Week-1	18
4.2 Activity log for the Second week	19
4.2.1 Weekly Report of Week-2	20
4.3 Activity log for the Third week	21
4.3.1 Weekly Report of Week-3	22
4.4 Activity log for the Fourth week	23
4.4.1 Weekly Report of Week-4	24
4.5 Activity log for the Fifth week	25
4.5.1 Weekly Report of Week-5	26
4.6 Activity log for the Sixth week	27
4.6.1 Weekly Report of Week-6	28
4.7 Activity log for the Seventh week	29
4.7.1 Weekly Report of Week-7	30
4.8 Activity log for the Eighth week	31
4.8.1 Weekly Report of Week-8	32
CHAPTER 5: OUTCOMES DESCRIPTION	33
5.1 Work environment I have experienced	
5.2 Real time technical skills I have experienced	
Student Self Evaluation of the Short-Term Internship	34
Evaluation by the Supervisor of the Intern Organization	35
Photos & Video Links	36

CHAPTER 1: EXECUTIVE SUMMARY

The "Full Stack development with React Native" Summer Internship Program Provided By "Technical Hub Pvt. Ltd."

1.1 Learning Objectives of Summer Internship

- Gain Practical Knowledge
- Enhance Full Stack Development with React Native Skills
- Work on Real-Time Projects
- Develop Advanced Programming Skills

1.2 Learning Outcomes of Summer Internship

- Team Collaboration and Communication
- Continuous Learning and Adaptation
- Excellent Problem-Solving Knowledge
- Extensive Experience in Development

1.3 Summer Internship Summary

The summer internship at Technical Hub Pvt. Ltd. in the domain of Full Stack development with React Native was an enriching 8-week program. The internship provided hands-on experience in Full Stack Development, allowing the intern to apply theoretical knowledge in real-life situations. The intern had the opportunity to enhance their skills in ReactJs, ExpressJs, NodeJs, MongoDB, React Native and deepen their understanding of frontend frameworks and libraries. The program also focused on developing critical thinking abilities as the intern navigated through different user interface designs and user experience principles.

The intern had the opportunity to work on real projects, such as developing a responsive and dynamic website, providing valuable experience in working on software systems and understanding how they are implemented in a professional setting. The internship also provided an excellent opportunity to establish a tech network, connecting with influential people at the company and throughout the tech industry.

CHAPTER 2: OVERVIEW OF THE ORGANIZATION

2.1 Introduction of the Organization

Technical Hub Pvt. Ltd., provides the trainings which are certification aligned and associated with various vendors that are top in the industry. Members who are part of this program are given a constant practical example on various topics, thus keeping them abreast of their technological world. It is located in Surampalem, Andhra Pradesh. Founded by Babji Neelam, Technical Hub Pvt.Ltd., quickly grew into one of the largest companies in the Kakinada. Today, Technical Hub Pvt. Ltd is one of the largest technology Educator in Kakinada District.

2.2 Mission of the Organization

Technical Hub Pvt.Ltd, the Student Success Platform Improving the knowledge base with rapidly evolving technological skills, the skill to communicate across discipline, the ability to lead team-centered projects, contextualized problem formulation and hand-on experience is the present demand of the global industry. To meet this demand, Aditya University initiated t-Hub: a perfect launch pad to the job world. T-Hub trains students in various disciplines beyond technological labels.

2.3 Vision of the Organization

We believe that the business of business is to improve the state of the world, and we work to make sure Technical Hub Pvt. Ltd is a platform for change through serving the interests of all our stakeholders — employees, customers, partners, communities and the environment.

CHAPTER 3: INTERNSHIP PART

3.1 A Discourse on Full Stack Development

The discipline of Full Stack Development is formally characterized by its comprehensive engagement with both the client-side and server-side strata of a web application's architecture. A practitioner in this domain, designated as a full stack developer, is required to possess a multifaceted and versatile skill set, which facilitates the construction of a complete, end-to-end software solution ab initio. This scope of responsibility encompasses the creation of the graphical user interface with which end-users interact, the administration of the server and database infrastructure that underpins the application's functionality, and the establishment of seamless, efficient communication protocols between these distinct architectural layers.

Within the context of contemporary web engineering, such a holistic methodology is considered indispensable for the construction of applications that are robust, scalable, and computationally efficient. The focus of this professional practicum was concentrated upon a particularly prominent and powerful technological agglomeration for full stack development, known colloquially as the **MERN stack**.

3.1.1 An Exposition of the MERN Stack

The MERN stack is a technological framework, predicated entirely upon the JavaScript programming language, for the development of web applications. The designation itself is an acronym, representing the four keystone technologies that constitute its foundational core:

- **MongoDB:** A non-relational, document-oriented database system employed for the persistence of application data. Data is stored in a flexible, JSON-like document format, the schema of which renders it particularly well-suited for modern, data-intensive applications demanding considerable scalability and adaptability.
- **Express.js:** A minimalist and highly flexible web application framework for Node.js, which furnishes a robust set of functionalities for the construction of backend Application Programming Interfaces (APIs). Its primary function is to simplify the complex processes of request handling, URI routing, and middleware integration.
- **React:** A declarative and potent JavaScript library, under the stewardship of Facebook, engineered for the construction of dynamic and interactive user interfaces. It employs a

component-based architectural paradigm, thereby permitting developers to create encapsulated, reusable interface elements.

- **Node.js:** A server-side JavaScript runtime environment, which facilitates the execution of JavaScript code outside the confines of a web browser. It is instrumental in the creation of performant, scalable, and efficient network services.

A principal advantage afforded by the MERN stack is the uniform application of a single programming language, JavaScript, across the entirety of the application's architecture. This technological homogeneity serves to streamline the development lifecycle, potentially reduce the cognitive load associated with learning disparate languages, and foster enhanced synergy between frontend and backend engineering teams.

3.2 Configuration of the Development Environment

An antecedent and critical phase in any software engineering endeavor is the establishment of a robust and efficient development environment. This procedure ensures the correct installation and configuration of all requisite tooling, thereby enabling a workflow characterized by maximal productivity and minimal friction.

3.2.1 Installation of Core Instrumentation

1. Node.js and npm (Node Package Manager): The Node.js runtime environment constitutes the foundational platform for the MERN stack's backend. The installation of Node.js concurrently provides the Node Package Manager (npm), a dependency management utility of paramount importance for the administration of project libraries and packages.

- **Procurement:** The Long Term Support (LTS) version was procured from the official digital repository located at the nodejs.org domain.
- **Verification:** Subsequent to the installation procedure, a terminal or command-line interface was invoked, whereupon the commands `node -v` and `npm -v` were executed to ascertain the successful installation and to report the respective version numbers.

2. Visual Studio Code (VS Code): Visual Studio Code is a source code editor of notable power and efficiency, developed and maintained by Microsoft. It has achieved a de facto standard status within the web development community, attributable to its extensive feature set, profound customizability, and a vast ecosystem of third-party extensions.

- **Requisite Extensions:**

- **Prettier - Code formatter:** For the automated enforcement of a consistent and uniform code style.
- **ES7+ React/Redux/React-Native snippets:** To expedite the generation of boilerplate code constructs.
- **Live Server:** To instantiate a local development server featuring live-reloading capabilities for frontend development.

3. Git and GitHub: The utilization of a version control system is considered a non-negotiable tenet of modern software development practices.

- **Git:** A distributed version control system designed for the meticulous tracking of alterations in source code over time.
- **GitHub:** A cloud-based hosting platform for Git repositories, which provides tooling for collaborative code development and management.
 - A GitHub account was provisioned, and the local Git installation was configured with user credentials to facilitate the pushing of project source code to a remote repository.

4. Postman: During the construction of a backend API, the availability of a specialized tool for testing service endpoints, independent of a graphical frontend, is a matter of considerable importance. Postman is an API development platform that permits the formulation and transmission of HTTP requests (e.g., GET, POST, PUT, DELETE) to a server and the subsequent inspection of the server's responses, thereby significantly streamlining the API development and debugging workflow.

3.3 Backend Development utilizing Node.js and Express.js

The backend, otherwise known as the server-side, functions as the computational engine of a web application. Its responsibilities include the execution of business logic, interaction with the data persistence layer, and the provisioning of data to the frontend via an Application Programming Interface (API).

3.3.1 An Introduction to the Node.js Runtime

Node.js is a runtime environment that enabled JavaScript to transcend its traditional client-side limitations and be executed on the server. Its architectural design, which is non-blocking and event-

driven, confers a high degree of efficiency, rendering it an exemplary choice for the construction of scalable network applications. An examination of its core modules was undertaken, including the `http` module for the creation of web servers and the `fs` module for filesystem interactions.

3.3.2 The Construction of a RESTful API with Express.js

Express.js is a framework that operates atop Node.js, providing a higher level of abstraction that dramatically simplifies the complexities associated with building a web server. It was employed in this practicum to create a RESTful API, an architectural style that has become a standard for designing networked applications.

1. Project Initialization: Within a newly created project directory, the command `npm init -y` was executed to generate a `package.json` file. Subsequently, the Express framework was installed via the execution of `npm install express`.

2. Instantiation of a Basic Server: A file designated `server.js` was created, into which the following code was inscribed to configure a rudimentary server:

```
const express = require('express');

const app = express();

const port = 5000;

app.get('/', (req, res) => {

  res.send('Hello, World!');

});

app.listen(port, () => {

  console.log(`Server running on http://localhost:${port}`);

});
```

The resultant server is configured to listen for HTTP GET requests directed at the root URI (`/`) and to respond with the string literal "Hello, World!".

3.3.3 A Conceptual Overview of Middleware

Middleware functions represent a central architectural concept within the Express framework. These are functions that possess access to the request object (req), the response object (res), and the subsequent middleware function in the application's request-response pipeline, conventionally denoted as next. Such functions are capable of executing arbitrary code, effecting modifications to the request and response objects, terminating the request-response cycle, or delegating control to the next function in the middleware stack.

Middleware was utilized for a variety of tasks, including:

- **express.json():** For the parsing of incoming request bodies formatted in JSON.
- **CORS (Cross-Origin Resource Sharing):** To permit the frontend, operating on a different origin (port), to issue requests to the backend server.
- **Authentication:** To safeguard specific routes by means of verifying user credentials.

3.3.4 Request Routing

With the progressive expansion of the application's functional scope, the API endpoints were organized into modular routing files. As an example, all user-centric endpoints (e.g., /api/users/register, /api/users/login) were consolidated within a users.js file, located in a dedicated routes directory. This practice maintains the cleanliness of the primary server.js file and enhances the overall maintainability of the API.

3.4 Database Administration with MongoDB

MongoDB serves as the data persistence component of the MERN stack. In contrast to traditional SQL databases, which employ a rigid, tabular structure of rows and columns, MongoDB is a NoSQL database that stores data within flexible, JSON-like documents.

3.4.1 An Introduction to NoSQL Database Systems

The initial phase of this module involved developing an understanding of the salient distinctions between SQL and NoSQL database paradigms. NoSQL databases are frequently selected for their notable advantages, which include:

- **Flexible Data Models:** The absence of a mandatory, predefined schema.

- **Horizontal Scalability:** The capacity to manage voluminous datasets through distribution across numerous servers.
- **High Performance:** An optimization for particular data models and access patterns.

3.4.2 MongoDB Atlas and Mongoose

- **MongoDB Atlas:** A fully-managed, cloud-hosted database service. A complimentary cluster was provisioned on MongoDB Atlas to serve as the database for the application, an action that obviated the considerable complexities associated with the setup and ongoing management of a database server.
- **Mongoose:** An Object Data Modeling (ODM) library specifically designed for MongoDB and Node.js. It furnishes a straightforward, schema-based methodology for modeling application data. Mongoose is instrumental in managing data relationships, providing schema validation, and mediating the translation between JavaScript objects in the application code and their corresponding document representations within MongoDB.

3.4.3 The Creation of Schemas and Models

Utilizing Mongoose, a Schema was defined for the application's data structures. A schema corresponds to a MongoDB collection and formally defines the shape and constraints of the documents contained therein.

Exemplar: A User Schema

```
const mongoose = require('mongoose');

const Schema = mongoose.Schema;

const UserSchema = new Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
```

```

    required: true,

    unique: true

  },

  password: {

    type: String,

    required: true

  },

  date: {

    type: Date,

    default: Date.now

  }

});

module.exports = User = mongoose.model('users', UserSchema);

```

A Model is a constructor that is compiled from a Schema definition. An instantiation of a model represents a single document that is capable of being persisted to and retrieved from the database.

3.4.4 CRUD Operations

With the data models formally defined, the core **CRUD (Create, Read, Update, Delete)** operations were implemented within the API routes. In the context of the user registration route, for instance, a new user document would be created and subsequently saved to the database.

3.5 Frontend Development with the React Library

React is a JavaScript library designed for the construction of user interfaces. Its component-based architectural model permits the assembly of complex user interfaces from small, isolated, and reusable units of code known as "components."

3.5.1 Core Concepts of React

1. Components: Components constitute the fundamental units of a React application. The implementation primarily utilized **Functional Components** in conjunction with **Hooks**. A component can be conceptualized as a JavaScript function that returns a fragment of the user interface, expressed using JSX.

2. JSX (JavaScript XML): JSX is a syntactic extension to JavaScript that bears a strong resemblance to XML or HTML. It provides a declarative syntax for describing the structure of the user interface, which is subsequently transpiled by React into JavaScript objects.

3. State and Props:

- **State:** Data that is managed internally by a component. A change in a component's state precipitates a re-rendering of that component to reflect the updated data. State was managed programmatically through the utilization of the `useState` Hook.
- **Props (Properties):** Data that is passed unidirectionally from a parent component to a child component. Props are immutable from the perspective of the receiving component.

4. The Virtual DOM: React employs a "Virtual DOM" as a performance optimization strategy. Rather than directly manipulating the browser's Document Object Model (DOM), React maintains a lightweight, in-memory representation thereof. When a component's state is altered, React first updates this Virtual DOM. It then performs a comparison, or "diffing," algorithm against a pre-update snapshot of the Virtual DOM. Finally, it applies only the necessary, minimal changes to the actual browser DOM. This process, termed "reconciliation," is substantially more performant than re-rendering the entire document tree.

3.5.2 Initialization of a React Project

The Create React App command-line utility was employed to initialize a new React single-page application. This tool automates the configuration of a complex build pipeline. `npx create-react-app client`

Execution of this command results in the creation of a client directory containing a standardized React project structure, replete with all requisite dependencies and operational scripts (e.g., `npm start`, `npm run build`).

3.5.3 State Management utilizing the Context API and Redux

For applications of a certain complexity, the management of state that must be shared across disparate and deeply nested components can become a significant architectural challenge, a phenomenon often referred to as "prop drilling." An exploration of two prominent solutions for global state management was conducted:

- **Context API:** A feature natively integrated into React for the purpose of sharing state without the necessity of manual prop-passing through every level of the component hierarchy. It is particularly well-suited for managing ambient concerns such as theming (e.g., dark/light mode) or user authentication status.
- **Redux:** A predictable state container for JavaScript applications. It provides a more structured and robust paradigm for the management of global state, especially within the context of large-scale applications. Its architecture is centered around a singular "store" and employs "actions" and "reducers" to govern state transitions in a strictly defined and predictable manner.

3.6 An Introduction to the React Native Framework

Subsequent to the establishment of a solid foundation in web development with the MERN stack, the practicum transitioned to the domain of mobile application development, employing the **React Native** framework.

3.6.1 A Definition of React Native

React Native is a software framework, also developed by Facebook, for the construction of native mobile applications using JavaScript and the React library. The operative term in this definition is "native mobile applications." In contrast to web-based frameworks that produce applications rendered within a web view, React Native utilizes the same fundamental user interface building blocks as conventional iOS and Android applications. The developer writes code in JavaScript, and the React Native framework subsequently invokes the native rendering APIs in Objective-C (for iOS) or Java (for Android).

This "write once, run anywhere" paradigm allows for the use of a singular codebase to generate applications for both major mobile platforms, which may yield substantial efficiencies in terms of development time and resource allocation.

3.6.2 Salient Distinctions: React versus React Native

Although the core principles (e.g., components, state, props) are largely congruent, several critical distinctions exist:

- **Rendering:** React leverages standard web technologies such as HTML and CSS (e.g., `<div>`, `<span>`, `<p>`) to render components within a web browser. React Native, conversely, provides its own library of built-in components that are mapped directly to native platform UI widgets (e.g., `<View>`, `<Text>`, `<Image>`).
- **Styling:** Within the React Native environment, styling is accomplished programmatically using JavaScript. Styles are defined as objects, and their properties are analogous to CSS, but employ camelCase naming conventions (e.g., `backgroundColor` in lieu of `background-color`). Separate CSS files are not utilized.
- **Navigation:** Web applications typically employ libraries such as React Router for navigation. Mobile applications, however, feature distinct navigational paradigms (e.g., stack, tab, drawer), and consequently necessitate the use of specialized libraries, such as **React Navigation**, for screen management.

3.6.3 Configuration of the React Native Environment

The development environment was configured utilizing the **Expo CLI**. Expo is a framework and platform for universal React applications, providing a suite of tools and services built upon React Native that serves to substantially lower the barrier to entry.

1. Installation of Expo CLI: `npm install -g expo-cli`

2. Creation of a New Project: `expo init MyFirstApp`

3. Application Execution: `cd MyFirstApp npm start`

Execution of this command initiates the Metro Bundler and opens a new browser tab displaying the Expo Developer Tools. From this interface, the application could be executed on several targets:

- An iOS simulator (which necessitates macOS and Xcode).
- An Android emulator (which necessitates Android Studio).

- A physical device, by scanning a provided QR code with the **Expo Go** mobile application.

3.7 The Construction of a Mobile Application with React Native

The knowledge acquired from React development was applied to the construction of a mobile application, with a particular focus on the unique considerations and constraints of the mobile environment.

3.7.1 Core Components in React Native

Proficiency was developed in the use of the fundamental components furnished by the React Native framework:

- **<View>**: The most elemental component for UI layout construction. It serves as a container that supports Flexbox-based layout, styling, touch handling, and accessibility controls, and is functionally analogous to the `<div>` element in web development.
- **<Text>**: A component for the display of textual content. It is a mandatory container for all text within a React Native application.
- **<Image>**: A component for the display of various image types, including network-fetched images, static application resources, and temporary local images.
- **<TextInput>**: A foundational component for the input of text into the application via a virtual or physical keyboard.
- **<ScrollView>**: A generic, scrollable container capable of hosting multiple components and views.
- **<TouchableOpacity>**: A wrapper component that provides user feedback for touch events by reducing the opacity of the wrapped view upon press.

3.7.2 Styling in React Native

Styling in React Native is implemented through the StyleSheet API.

```
import { StyleSheet, Text, View } from 'react-native';

const App = () => (

  <View style={styles.container}>

    <Text style={styles.title}>Hello, Native World!</Text>
```

```
</View>

);

const styles = StyleSheet.create({

  container: {

    flex: 1,

    padding: 24,

    backgroundColor: '#eaeaea',

    justifyContent: 'center',

    alignItems: 'center',

  },

  title: {

    marginTop: 16,

    paddingVertical: 8,

    borderWidth: 4,

    borderColor: '#20232a',

    borderRadius: 6,

    backgroundColor: '#61dafb',

    color: '#20232a',

    textAlign: 'center',

    fontSize: 30,

    fontWeight: 'bold',

  },

});
```

3.7.3 Layout with Flexbox

React Native employs **Flexbox** as its primary layout engine. The Flexbox algorithm is designed to provide a consistent and predictable layout across a heterogeneous range of screen sizes. Its properties function in a manner similar to CSS Flexbox, albeit with certain different default values. For instance, the default flexDirection is column in React Native, whereas it is row in standard web CSS. A considerable amount of time was allocated to mastering Flexbox in order to produce responsive and adaptive layouts.

3.7.4 Navigation utilizing React Navigation

For the purpose of navigating between discrete screens within the mobile application, the **React Navigation** library was employed. This library is widely considered the most popular solution for routing and navigation in React Native applications. Instruction was received on the implementation of the three most prevalent navigation patterns:

- **Stack Navigator:** Manages screens within a last-in, first-out stack. New screens are pushed onto the top of the stack, and the back action pops them off.
- **Tab Navigator:** Renders a persistent tab bar, typically at the bottom of the screen, to facilitate switching between primary application routes.
- **Drawer Navigator:** Renders a navigational drawer that slides in from the edge of the screen, typically the left.

3.7.5 Accessing Native Device Features

A significant advantage of the React Native framework is its capacity to interface with native device APIs. By means of the Expo SDK, the integration of various features was accomplished with relative ease, including:

- **Camera:** For the capture of photographic images and video.
- **Location:** For the retrieval of the user's geographical coordinates.
- **Contacts:** For accessing the device's contact list.
- **SecureStore:** For the secure, encrypted storage of sensitive data on the device.

This capability permits the creation of rich, feature-complete applications that provide a user experience that is functionally and aesthetically native to the host platform.

3.8 Conclusion

Conclusion in development refers to the final stage of a project, program, or process aimed at achieving growth, progress, or positive change. It serves as the culmination of efforts, analysis, and implementation, offering a summary of the outcomes and insights gained throughout the developmental journey. A well-crafted conclusion typically includes key findings, lessons learned, and recommendations for future actions. It plays a crucial role in assessing the success of the development endeavor, highlighting achievements, and providing a roadmap for further improvements or initiatives. Ultimately, a thoughtful conclusion helps stakeholders and decision-makers make informed choices for ongoing development efforts and ensures that the development process is iterative and adaptive.

CHAPTER 4: WEEKLY REPORTS

4.1 Activity Log for the First Week

Day & Date	Brief Description of the Daily Activity	Learning Outcome	Signature
2025-06-02	Orientation for the practicum; introduction to Full Stack Development and the MERN stack.	Comprehension of practicum objectives and the functional roles of MongoDB, Express, React, and Node.js was established.	
2025-06-03	Configuration of the development environment, including Node.js, npm, VS Code, and Git.	All core development tools were proficiently installed and configured.	
2025-06-04	Introduction to Node.js; construction of a basic web server utilizing the http module.	The fundamental principles of Node.js and its event-driven architecture were assimilated.	
2025-06-05	Introduction to the Express.js framework, encompassing server setup and basic routing.	The ability to instantiate a simple Express.js server and handle GET requests was acquired.	
2025-06-06	In-depth examination of Express middleware and the request-response lifecycle.	An understanding of how to employ middleware for logging, parsing, and Cross-Origin Resource Sharing was developed.	
2025-06-07	Review of Week 1 activities; establishment of a project repository on GitHub.	Learning was consolidated, and proficiency with fundamental Git commands was achieved.	

4.1.1 WEEKLY REPORT FOR WEEK - 1

WEEK – 1 (From Dt: 02/06/2025 to Dt: 07/06/2025)

Objective: The primary objective for this period was to establish a foundational understanding of the MERN stack, to configure a complete development environment, and to attain mastery of the basic principles involved in creating a backend server with Node.js and Express.js.

Detailed Summary: The inaugural week of the practicum was dedicated to the establishment of a robust foundational knowledge base. The period commenced with an orientation session that delineated the program's objectives and furnished a high-level overview of Full Stack Development with the MERN stack. The local development environment was successfully configured through the installation of Node.js, npm, Visual Studio Code with requisite extensions, and Git. Core technical instruction began with Node.js, wherein the method for creating a basic HTTP server was learned. A rapid transition was made to the Express.js framework, which simplifies server creation. By the conclusion of the week, the capacity to construct a simple Express server, define routes for request handling, and comprehend the critical function of middleware in processing incoming requests had been attained. The week culminated in a review session and the creation of a GitHub repository for the project, which provided an opportunity to practice essential Git commands such as init, add, commit, and push.

4.2 Activity Log for the Second Week

Day & Date	Brief Description of the Daily Activity	Learning Outcome	Signature
2025-06-09	Introduction to NoSQL databases; configuration of a MongoDB Atlas cluster.	An understanding of the advantages of NoSQL was gained, and a cloud database was successfully provisioned.	
2025-06-10	Introduction to Mongoose; connecting to MongoDB and defining data schemas.	The ability to connect a Node.js application to MongoDB and define data structures via schemas was acquired.	
2025-06-11	Implementation of Mongoose Models; execution of CRUD operations (Create & Read).	API endpoints were authored to facilitate the creation and retrieval of data from the database.	
2025-06-12	Implementation of CRUD operations (Update & Delete).	The backend API was completed, endowing it with full data manipulation capabilities.	
2025-06-13	Structuring Express routes for modularity using express.Router().	The technique of organizing API endpoints into discrete files for enhanced maintainability was learned.	
2025-06-14	Comprehensive testing of all backend API endpoints utilizing Postman.	Proficiency was gained in the domain of API testing and the debugging of server responses.	

4.2.1 WEEKLY REPORT FOR WEEK - 2

WEEK – 2 (From Dt: 09/06/2025 to Dt: 14/06/2025)

Objective: The objective for this period was to integrate a data persistence layer into the backend by learning MongoDB and Mongoose, and to construct a complete, testable RESTful API endowed with full CRUD functionality.

Detailed Summary: The second week was focused on the MongoDB component of the MERN stack. An introduction to the concepts of NoSQL databases was provided, and a complimentary cluster was configured on the MongoDB Atlas platform to host the application's data. Subsequently, the Mongoose library was integrated into the Express application to serve as an abstraction layer for database communication. The core Mongoose concepts of Schemas, for defining data structure, and Models, for interacting with a data collection, were mastered. The principal focus of the week was the implementation of API endpoints for all four CRUD (Create, Read, Update, Delete) operations. Routes were successfully created to manage user registration, data retrieval, record updates, and deletions. To improve the architectural quality of the codebase, the `express.Router()` utility was employed to modularize the routing logic. The week concluded with the extensive and systematic testing of every API endpoint using the Postman application to verify correct functionality prior to proceeding to frontend development.

4.3 Activity Log for the Third Week

Day & Date	Brief Description of the Daily Activity	Learning Outcome	Signature
2025-06-16	Introduction to React; core concepts including Components, JSX, and Props.	The fundamental architectural constructs of a React application were understood.	
2025-06-17	Initialization of a React project using the create-react-app utility.	A new single-page application was successfully bootstrapped.	
2025-06-18	State management in React utilizing the useState Hook.	The methodology for managing component-level state and precipitating re-renders was learned.	
2025-06-19	Handling of events and forms within the React paradigm.	The ability to construct interactive forms and manage user input was acquired.	
2025-06-20	Conditional rendering and the rendering of data lists.	Proficiency was achieved in the dynamic display of UI elements based on application state.	
2025-06-21	Review of Week 3 activities; construction of a simple single-page user interface.	React concepts were consolidated through the construction of a small, self-contained frontend application.	

4.3.1 WEEKLY REPORT FOR WEEK - 3

WEEK - 3 (From Dt: 16/06/2025 to Dt: 21/06/2025)

Objective: The objective for this period was to acquire fundamental knowledge of frontend development with the React library and to construct a basic user interface.

Detailed Summary: This week signified the transition to frontend development with React. The instruction commenced with the core concepts, fostering an understanding of React's component-based architecture, its use of JSX for templating, and the props mechanism for data transmission. A new React project was initialized using the create-react-app command-line interface tool. A substantial portion of the week was allocated to achieving mastery of state management within components through the useState Hook, a concept that is fundamental to the creation of dynamic user interfaces. The methodologies for handling user-generated events, such as button clicks and form submissions, and for controlling form inputs within a React context were covered. Furthermore, the techniques of conditional rendering, for the selective display of UI elements based on state, and the rendering of dynamic data lists using the .map() array method were examined. The week's activities culminated in the construction of a small but complete user interface that synthesized all of the concepts learned.

4.4 Activity Log for the Fourth Week

Day & Date	Brief Description of the Daily Activity	Learning Outcome	Signature
2025-06-23	Utilization of the useEffect Hook for handling side effects (e.g., data fetching).	The method for fetching data from an API upon component mounting was learned.	
2025-06-24	Connection of the React frontend to the backend API utilizing the Axios library.	Data was successfully fetched from the Express API and displayed within the user interface.	
2025-06-25	Implementation of client-side routing with the React Router library.	The ability to simulate a multi-page experience within a single-page application was acquired.	
2025-06-26	Global state management utilizing the Context API and preventing common vulnerabilities	An understanding was gained of how to share state, such as user authentication status, across the application.	
2025-06-27	Full MERN stack integration: Connection of registration and login forms to the API.	A complete authentication workflow was constructed, spanning from the frontend to the backend.	
2025-06-28	Styling of the React application with CSS and a component library (e.g., Material-UI).	The techniques for creating a visually appealing and professional user interface were learned.	

4.4.1 WEEKLY REPORT FOR WEEK - 4

WEEK - 4(From Dt: 23/06/2025 to Dt:28/06/2025)

Objective: The objective for this period was to integrate the React frontend with the Express backend, thereby creating a full end-to-end MERN stack application complete with routing and global state management.

Detailed Summary: During the fourth week, the two primary architectural components of the application were integrated. The useEffect Hook was introduced as the standard mechanism for executing side effects, with a primary application being the fetching of data from the backend API. The Axios library was employed to facilitate asynchronous HTTP requests from the React application to the Express server. A significant milestone was achieved with the successful display of data retrieved from the MongoDB database within the React user interface. Subsequently, client-side routing was implemented using the React Router library, which enables navigation between different application views without necessitating a full page reload. To manage application-scoped state, such as user authentication details, the Context API was studied and implemented. By the conclusion of the week, a fully integrated MERN application had been realized. A user could be registered via a React form, with the corresponding data persisted to MongoDB, and subsequently authenticated, thus creating a complete authentication system. Finally, attention was given to the application's aesthetic and professional presentation through the application of CSS and the use of a UI component library.

4.5 Activity Log for the Fifth Week

Day & Date	Brief Description of the Daily Activity	Learning Outcome	Signature
2025-06-30	Introduction to Mobile Development and the React Native framework.	An understanding of the benefits of cross-platform development with React Native was gained.	
2025-07-01	Configuration of the React Native environment utilizing the Expo CLI.	Expo was successfully installed, and a basic application was executed on a device or emulator.	
2025-07-02	Study of Core Components: <View>, <Text>, <Image>, <TextInput>.	Knowledge of the fundamental building blocks for constructing a mobile user interface was acquired.	
2025-07-03	Styling in React Native with the StyleSheet API.	The ability to apply styles to components through the use of JavaScript objects was developed.	
2025-07-04	Layout implementation with Flexbox in React Native.	Mastery of Flexbox was achieved for the purpose of creating responsive layouts adaptable to various screen sizes.	
2025-07-05	Review of Week 5 activities; construction of a simple static mobile screen.	Core React Native concepts were consolidated through the construction of a single-screen user interface.	

4.5.1 WEEKLY REPORT FOR WEEK - 5

WEEK - 5 (From Dt: 30/06/2025 to Dt: 05/07/2025)

Objective: The objective for this period was to transition from web to mobile development, to comprehend the fundamental principles of React Native, and to construct a basic mobile user interface.

Detailed Summary: The fifth week occasioned a strategic pivot from web development to mobile application development with React Native. The period began with an introduction to the framework, elucidating how it leverages React principles to build truly native applications for both iOS and Android from a singular codebase. The development environment was configured using the Expo CLI, a toolchain that substantially simplifies the setup process. A first application was successfully executed on a physical device by means of the Expo Go application. The remainder of the week was devoted to assimilating the fundamental building blocks of React Native. Instruction covered the use of core components such as <View> and <Text> as replacements for their web-based analogues, <div> and <p>. The methodology for styling via the StyleSheet API was examined, and considerable time was invested in mastering layout with Flexbox, the primary layout engine in React Native. The week's work concluded with the construction of a simple, static mobile screen that integrated all of these newly acquired concepts.

4.6 Activity Log for the Sixth Week

Day & Date	Brief Description of the Daily Activity	Learning Outcome	Signature
2025-07-07	Handling touch events with the <TouchableOpacity> and <Button> components.	The methods for endowing a mobile application with interactivity were learned.	
2025-07-08	Construction of data lists with the <FlatList> and <ScrollView> components.	An understanding was gained of how to efficiently render long, scrollable lists of data.	
2025-07-09	Introduction to navigation utilizing the React Navigation library.	The basic principles of navigating between screens within a mobile application were learned.	
2025-07-10	Implementation of a Stack Navigator for managing screen transitions.	The ability to create a fundamental navigation flow, wherein screens are pushed onto and popped from a stack, was acquired.	
2025-07-11	Construction and validation of forms using the <TextInput> component.	The techniques for creating forms to capture user input in a mobile context were learned.	
2025-07-12	Connection of the React Native application to the MERN backend API.	Data was successfully fetched from the existing API and rendered within the mobile application.	

4.6.1 WEEKLY REPORT FOR WEEK - 6

WEEK - 6 (From Dt: 07/07/2025 to Dt: 12/07/2025)

Objective: The objective for this period was to construct an interactive, multi-screen React Native application and to establish a connection with the existing backend API.

Detailed Summary: The focus of this week was on rendering the mobile application functional and interactive. The initial task was to learn the handling of user touch and press events through components such as `<TouchableOpacity>`. A crucial aspect of many mobile applications is the display of data lists; the efficient rendering of such lists was accomplished using the `<FlatList>` component. Subsequently, one of the most critical concepts in mobile development, navigation, was addressed. The React Navigation library was installed, and a Stack Navigator was implemented to permit users to transition between different application screens. Forms were also constructed using the `<TextInput>` component to capture user input. The major achievement of the week was the successful connection of the React Native frontend to the MERN backend API that was developed in preceding weeks. The Axios library was utilized to fetch data from the server and display it within the mobile application's interface.

4.7 Activity Log for the Seventh Week

Day & Date	Brief Description of the Daily Activity	Learning Outcome	Signature
2025-07-14	Implementation of a Tab Navigator for primary application sections.	A different style of navigation, common in mobile applications, was learned and implemented.	
2025-07-15	Passing data between screens through the use of route parameters.	An understanding of how to transmit information to a new screen during a navigational transition was acquired.	
2025-07-16	Introduction to the Expo SDK for accessing native device APIs.	The utility of the Expo framework for accessing device features without requiring ejection was learned.	
2025-07-17	Utilization of the device Camera via the expo-camera module.	A feature was implemented to capture a photograph using the device's integrated camera.	
2025-07-18	Retrieval of device location coordinates via the expo-location module.	A feature was implemented to fetch and display the user's current geographical coordinates.	
2025-07-19	Secure local data storage on the device using the expo-secure-store module.	The methodology for persisting sensitive data, such as authentication tokens, was learned.	

4.7.1 WEEKLY REPORT FOR WEEK - 7

WEEK - 7 (From Dt: 14/07/2025 to Dt: 19/07/2025)

Objective: The objective for this period was to enhance the mobile application with advanced navigation patterns and to integrate native device features by means of the Expo Software Development Kit (SDK).

Detailed Summary: In the seventh week, an exploration of more advanced mobile development concepts was undertaken. A Tab Navigator was implemented using the React Navigation library to create a primary menu at the bottom of the screen, a very common user interface pattern. The mechanism for passing data between screens—for instance, navigating from a list item to a detail screen for that item—was also learned and implemented. The most significant portion of the week's work involved an investigation of the Expo SDK for the purpose of accessing native device functionality. Several modules were examined, leading to the successful implementation of features to activate the device camera, retrieve the user's current GPS location, and securely store an authentication token on the device using expo-secure-store. This week served to demonstrate the power of React Native and Expo in creating feature-rich applications capable of interacting with the device's hardware.

4.8 Activity Log for the Eighth Week

Day & Date	Brief Description of the Daily Activity	Learning Outcome	Signature
2025-07-21	Finalization of the mobile application's UI/UX; ensuring design consistency.	The application's aesthetic and user experience were enhanced.	
2025-07-22	End-to-end testing of the full stack application, including web and mobile clients.	Comprehensive testing was performed to identify and remediate software defects.	
2025-07-23	Preparation for deployment: Creation of a production build for the React web application.	The process of bundling and optimizing a web application for deployment was learned.	
2025-07-24	Building a standalone mobile application binary using the eas build command.	An understanding of the process for creating a distributable binary file for app stores was gained.	
2025-07-25	Preparation of the final project documentation.	Detailed documentation was authored, explaining the project's architecture and features.	
2025-07-26	Preparing the final project documentation and presentation	A presentation was created to showcase the project and summarize the learning outcomes of the practicum.	

4.1.1 WEEKLY REPORT FOR WEEK - 8

WEEK - 8 (From Dt: 21/07/2025 to Dt: 28/07/2025)

Objective: The objective for the final period was to finalize, test, and prepare both the web and mobile applications for deployment, and to complete all final deliverables for the practicum.

Detailed Summary: The final week of the practicum was allocated to the refinement of the project and the preparation of all required deliverables. A primary focus was the improvement of the User Interface (UI) and User Experience (UX) of both the web and mobile applications to ensure they were intuitive and aesthetically pleasing. A considerable quantum of time was devoted to comprehensive, end-to-end testing to identify and remediate any remaining defects in the full application workflow. Subsequently, the procedures for preparing an application for deployment were studied. For the React web application, this entailed the creation of an optimized production build. For the React Native application, the eas build service, provided by Expo, was utilized to generate standalone binary files (.apk or .ipa) suitable for submission to application marketplaces. The latter portion of the week was focused on the preparation of the formal project documentation and the final presentation. On the concluding day, the completed full stack project was presented, and the final internship report was submitted, thereby marking the conclusion of a challenging and highly instructive professional practicum.

CHAPTER 5: OUTCOMES DESCRIPTION

5.1 WORK ENVIRONMENT I HAVE EXPERIENCED

My summer internship at Technical Hub Pvt Ltd in Surampalem was a transformative experience. Immersed in a vibrant work environment, I honed my problem-solving skills and deepened my understanding of JavaScript. The guidance from mentors and the collaborative spirit of fellow interns were invaluable. This enriching journey not only covered all topics in JavaScript and Data Structures and Algorithms, but also competitive mathematics, paving the way for my future endeavors in the tech industry.

5.2 Real time technical skills I have experienced

During my summer internship at Technical Hub Pvt Ltd in Surampalem, I had the opportunity to apply and enhance my technical skills in a real-world setting. The focus on problem-solving and JavaScript programming allowed me to delve deeper into these areas. The comprehensive coverage of topics in JavaScript, Data Structures and Algorithms, and competitive mathematics equipped me with a robust skill set that is applicable in various technical domains..

Student Self-Evaluation of the Short-Term Internship

Student Name: CHITTUDI UMA MAHESWARA RAO

Registration No: 22P31A4405

Duration of Internship: 8 Weeks

From: 02-06-2025 **To:** 28-07-2025

Date of Evaluation:

Organization Name & Address: Technical Hub Private Limited

Andhra Pradesh, India - 533101

Please rate your performance in the following areas:

Please note that your evaluation shall be done independent of the student's self-evaluation

Rating Scale: 1 is the lowest and 5 is the highest rank

1	Oral communication	1	2	3	4	5
2	Written communication	1	2	3	4	5
3	Interaction ability with the community	1	2	3	4	5
4	Positive Attitude	1	2	3	4	5
5	Self-confidence	1	2	3	4	5
6	Ability to learn	1	2	3	4	5
7	Work Plan and Organization	1	2	3	4	5
8	Quality of work done	1	2	3	4	5
9	Time Management	1	2	3	4	5
10	Achievement of Desired Outcomes	1	2	3	4	5
OVERALL PERFORMANCE						

Student Signature

Evaluation by the Supervisor of the Intern Organization

Student Name: CHITTUDI UMA MAHESWARA RAO

Registration No: 22P31A4405

Duration of Internship: 8 Weeks

From: 02-06-2025 **To:** 28-07-2025

Date of Evaluation:

Organization Name & Address: Technical Hub Private Limited

Andhra Pradesh, India - 533101

Name of the Supervisor with Mobile Number:

Please rate the student's performance in the following areas:

Please note that your evaluation shall be done independent of the student's self-evaluation

Rating Scale: 1 is the lowest and 5 is the highest rank

1	Oral communication	1	2	3	4	5
2	Written communication	1	2	3	4	5
3	Interaction ability with the community	1	2	3	4	5
4	Positive Attitude	1	2	3	4	5
5	Self-confidence	1	2	3	4	5
6	Ability to learn	1	2	3	4	5
7	Work Plan and Organization	1	2	3	4	5
8	Quality of work done	1	2	3	4	5
9	Time Management	1	2	3	4	5
10	Achievement of Desired Outcomes	1	2	3	4	5
OVERALL PERFORMANCE						

Signature of the Supervisor

Photos of The Project

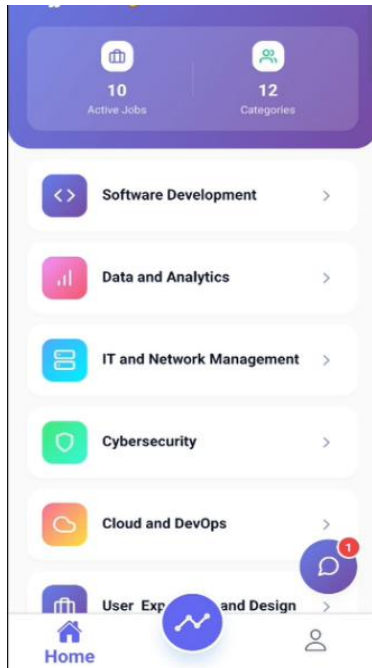


Fig -1 : Home page of App

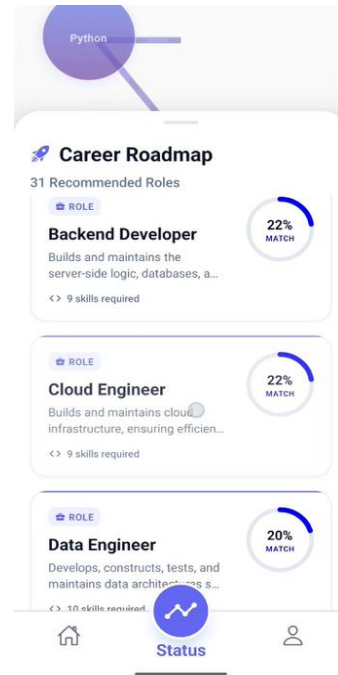


Fig -2 : Status Page

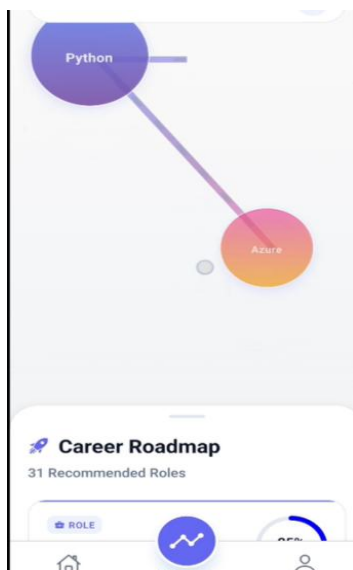


Fig -3 : Inner Screen of App

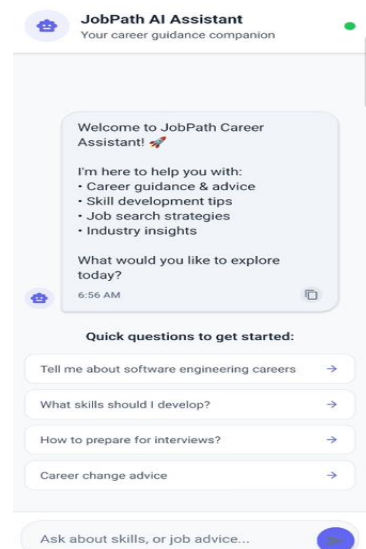


Fig -4 : Chatbot of App

